

Skill 4

Escape Sequences, Command History and Dynamic Memory Management

Aim

To implement escape-sequence handling, maintain command history, navigate through previously entered commands, dynamically allocate input buffers, resize arrays, manage linked lists and verify memory management using Valgrind.

Requirements

- Ubuntu Linux
- GCC
- Valgrind
- Terminal

Step-by-Step Implementation

Step 1: Install Valgrind

```
sudo apt install valgrind
```

Step 2: Handle Escape Sequences

Special keyboard sequences such as arrow keys generate escape sequences.

For example:

```
Up Arrow    → Previous command  
Down Arrow  → Next command
```

The program identifies these sequences and performs the corresponding operation.

Step 3: Maintain Command History

Each successfully entered command is stored in a history structure.

Example:

History:

1. `ls`
2. `pwd`
3. `gcc program.c`
4. `./program`

Step 4: Navigate Previous Commands

When the Up Arrow is pressed, the previous command is retrieved from the history.

Step 5: Navigate Next Commands

When the Down Arrow is pressed, the next command in the history is retrieved.

Step 6: Update the Input Buffer

The selected history command replaces the current input buffer.

Step 7: Allocate Memory Dynamically

Memory can be allocated using:

```
char *buffer = malloc(100 * sizeof(char));
```

Step 8: Resize the Buffer

When the input exceeds the allocated size:

```
buffer = realloc(buffer, new_size);
```

This allows larger commands to be stored.

Step 9: Manage Linked Lists

Commands can also be stored using a linked-list structure:

```
Command 1 → Command 2 → Command 3 → NULL
```

Step 10: Release Memory

Allocated memory must be released:

```
free(buffer);
```

Step 11: Verify Using Valgrind

Compile with debugging information:

```
gcc -g program.c -o program
```

Run:

```
valgrind --leak-check=full ./program
```

Observation

Escape sequences were used to detect navigation keys. Previously entered commands were stored and recalled using the Up and Down Arrow keys. Dynamic memory allocation and resizing allowed the program to handle variable-length input. Valgrind was used to detect memory leaks and invalid memory operations.

End Result

Command history, escape-sequence handling and dynamic memory management were successfully implemented and tested. Memory usage was verified using Valgrind.